流体网络计算调用说明

一、优点：

1、实现了解析运行：解决调试编译复杂、外部调用麻烦的问题，更容易地嵌入 3D、网页、云端运行。大多数情况下无需平台进行复杂编译、设计调试，仅需要传递“网络搭建拓扑关系”即可进行项目运算。

2、实现模块多线程并行运行：

①解决每秒迭代次数少的问题，小项目可实现 200 次每秒的迭代，很大项目可实现 30 次以上每秒的迭代。可实现燃气、暖通、市政、制冷行业的大规模数值模拟运算。

②解决“个别设备需要着重计算”的问题，计算量小的模块一堆运行，计算量大的模块单独运行，并通过调控进行同步启停，可解决燃气行业“压缩流体超高压泵”的模拟，更容易解决机械、气流组织等相关的 CFD 大规模运算方法的嵌入。

③此架构进行的电网计算，因省去了物性库的资源占用，有望达到 500 次以上每秒的迭代次数，可进行低频电网的高精度数值模拟，虽比不上 FPGA 专业的 2000 次每秒，但也有很大的进步，流体、电力、热力多专业融合有望真正实现，微电网计算技术正在开发验证。

3、实现了管路 “强制流量”的兼容计算：

①压力适应型强制流量：适用于制冷、空调、工业气泵等行业

的“容积型压缩机”的模拟，此种设备每秒输出的流量是由吸气量决定的，而非压差和阻力。

②阻力适应型强制流量：市政、燃气行业在进行城市管网模拟时，常需要将末端的流量进行设置，或是手动或是大数据自动分析设置，从而将市政采集的数据库加入到流网计算中，在此进行支持。

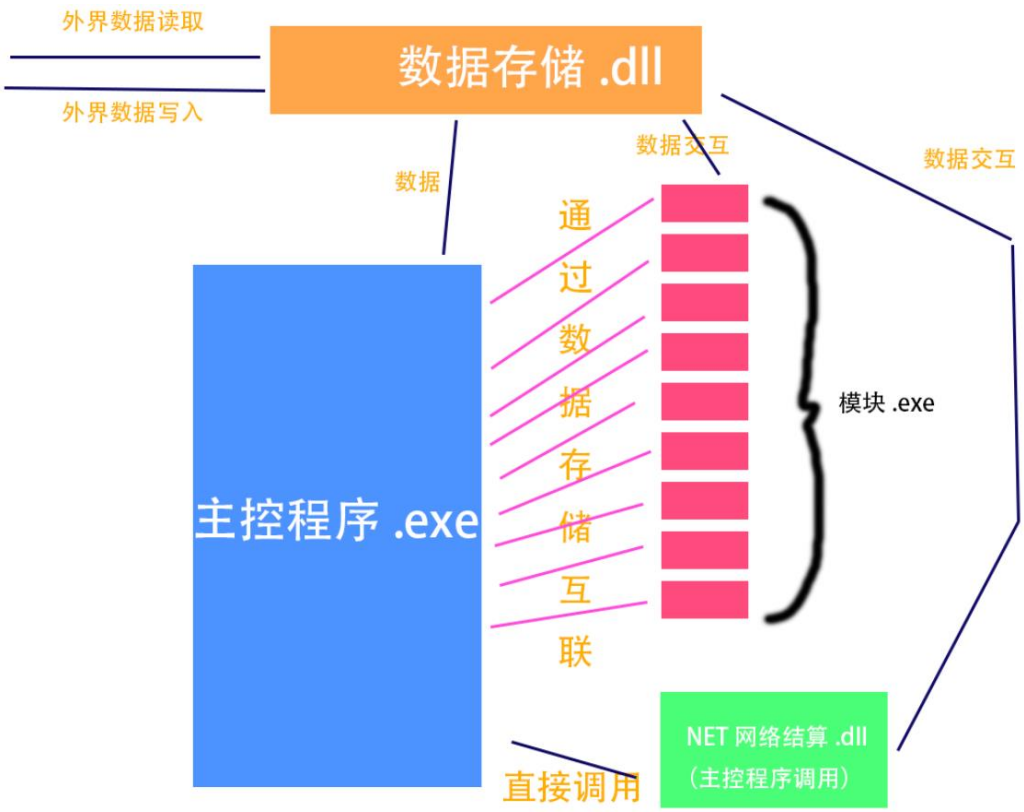
4、优化了“节点”的压力波动：有助于燃气“调压器”的相关联支路的稳定运行，更加真实的模拟调压器的工况以及调试、运行等操作的反馈。

5、实现了“可压缩”流体的长距离管道阻力精确计算：解决长输燃气管线、长输蒸汽热力管道的可变密度阻力计算。

6、解决了传统流网常见的问题之一：以两阀门之间的管道为对象，当两阀均关闭之后，压力转换为容积性质的计算。（专业一些的说法是“泄露属性的准确计算”），采用管网映射的方式，无缝进行多状态切换模拟，可用于燃气、蒸汽等管网的应急处理特性模拟。

二、多线程计算架构简述（多线程和单项目 exe 均兼容）

1、流体网络计算的内部结构如下图所示：分为主控程序、模块程序、存储库三部分。



① 主控程序调用负责接收界面端或者网络断发送来的组网信息，然后将组网信息解析并存储到数据储存 dll 中并开始运行。

② 主控程序控制先后运算顺序，顺序为模块 exe、net 的计算、模块 exe、net 的计算交替运行。

③ 模块 exe 之间并列运行，例如阀门模块和管道模块是独立运行的，分别依照 net.dll 分配的顺序,分别将一个项目中的阀门、管道进行解算。

④ 数据存储 dll 主要是存储每个模块的所有外置变量，以及存储主控程序的指令等，类似于 CPU L1 缓存的数据缓存。

⑤ NET.Dll 被调用后，除了解算之外，也用于存储网格拓扑关系，来指引模块 exe 的运行。

2、主控程序示例

这里面，主控程序其实是结构最简单的，如下图所示：

```
// NetCreative.cpp : 此文件包含 "main" 函数。程序执行将在此处开始并结束。
// NODEUL.cpp : 此文件包含 "main" 函数。程序执行将在此处开始并结束。
//

#include <iostream>
#include <windows.h>
#include "D:\Project(API)\DLLShear\VarLink.h"
#pragma comment(lib, "D:\\Project(API)\\DLLShear\\VarLink.lib")
#include "D:\Project(API)\DLLShear\FUDNET.h"
#pragma comment(lib, "D:\\Project(API)\\DLLShear\\FUDNET.lib")
#include "D:\Project(API)\DLLShear\FUDPHY.h"
#pragma comment(lib, "D:\\Project(API)\\DLLShear\\FUDPHY.lib")
int main()
{
    int xhxm = 0;
    while (xhxm < 0.5)
    {
        for (int iz = 0; iz < ProjectGet(); iz++)
        {
            if (NetCk(iz) > 0.5)
            {
                netmatrix(iz);
            }
            else
            {
                Sleep(2);
            }
        }
    }
}
```

① 调用 VarLinl.dll、FUDNET.DLL、FUDPHY.DLL

② 一个 for 循环，不断迭代运行

③ iz 是项目编号，本系统在同一文件目录下，支持多个项目并行。

④ 当检测到某一个项目的 NetCk 值为 1 时，证明此项目已经运行完所有模块的 exe，可以进行网络计算了，故开始流网解算运行。

⑤ 若没有检测到需要运行的流网解算，暂停 2 毫秒，此处可以有效减少等待时的资源消耗。

3、模块程序简述：以管道为例

① 自动检测并按照顺序执行某一项目的所有管道代码。

② 不同的管道是有不同的变量的，变量值均从数据存储.dll 中读取写入。

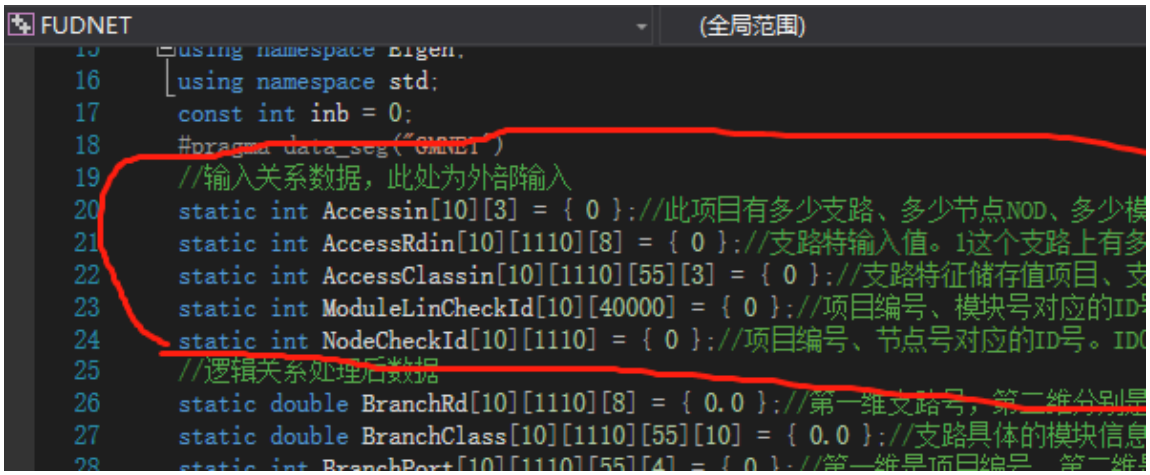
③ 每一个管道模块最后，需要调用流网的嵌入函数，即可并入到流网计算中，如下图所示：

```
}
//*************************************************************
// 计算与流体计算相关的值
//*************************************************************
temp = (PEO_TEMP + PLO_TEMP) / 2;
PIPECV(temp, DENSITY, X, PLO_FLOW, DN, PL, CDV, FLOW_SPD, ierr, herr);
pipe_embed(ProjectID,ModuleID, ModuleSort, TUNNEL, CDV, DENSITY, DHIGH, PLO_FLOW, PEO_PRE, PLO_PRE);
PEO_FLOW = PLO_FLOW;
//下面是赋值储存ccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
```

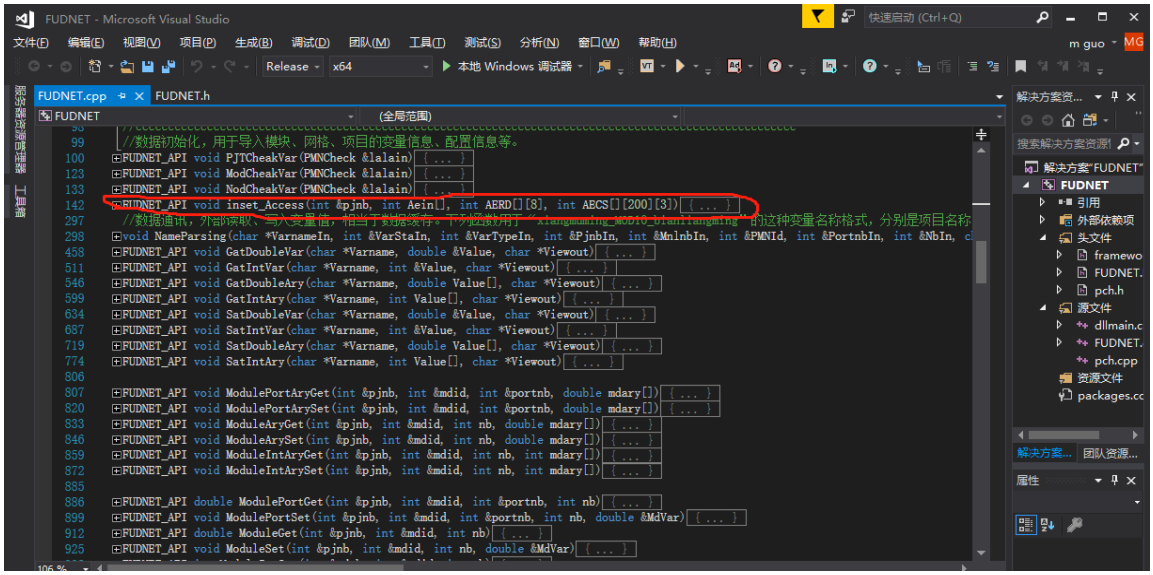
三、调用方法：网格解析并传输到流网

需要将连接的拓扑关系传递给流亡计算中，可以转化成 xml、GSV 等文档，或者直接转化为四个数组并调用 NET 的函数，输入到流体网络计算中。函数接口如下：

1、输入的数组如下：

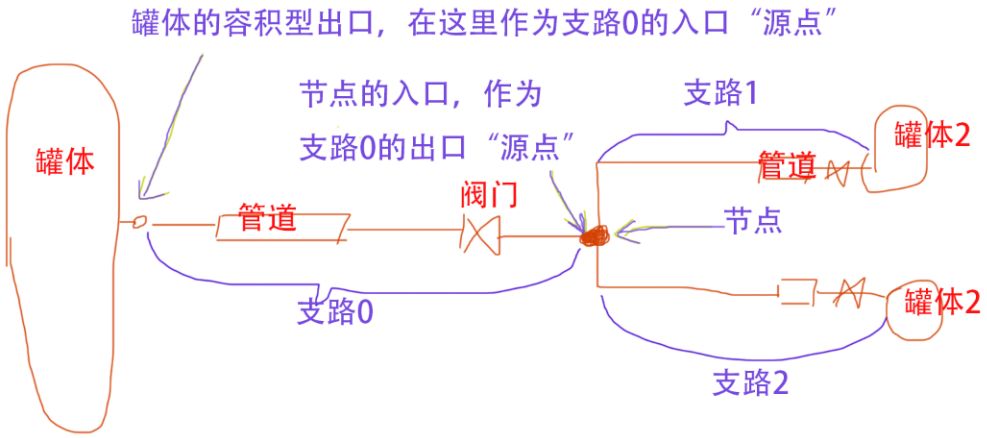


2、调用的解析函数如下：



3、连接关系分三个概念：①支路，就是很多道管道串联的一条路，支路号，即是分析出的支路的编号。②压力源点：容积型端口的压力，作为计算源点。③模块号，即界面自动为模块分配的编号，例如 MOD5、MOD6 等，端口号既是模块的端口的号。

4、图形化的支路、支路号、源点等的解释如下：



5、数组的意义解释：

① static int Accessin[10][3] = { 0 };//[10]代表项目编号，目前设置可同时并行10个项目。后面的[3]代表有多少支路、多少节点NOD、多少模块MOD。

② static int AccessRdin[10][1110][8] = { 0 };//[1110]这到表支

路号，此项目中有多少条支路、支路上有多少模块、上游节点是什么性质、下游链接节点是什么性质等信息。

③ static int AccessClassin[10][1110][55][3] = { 0 };//[1110]是支路号，[55]对应的是此支路上的单元，目前设置一条支路最多串联55个单元。[3]存储此单元的MOD模块号、通路号。

④ static int ModuleLinCheckId[10][40000] = { 0 };//[10]是项目号，[40000]代表本项目总共有多少个模块，目前设置单项目最高4万模块，此值为每一个模块号例如MOD5所对应的模块ID编号等。

6、上述规则，无论是 xml、还是直接调用函数，最终都是解析成第 5 条数组的内容信息。

四、数据的读取与写入

1、上述数组输入到函数后，流网会自动解析并运行。

2、平台、网站、云端、客户端等界面，若要调用运算结果，提供如下接口（可以依照平台性质修改）：

```
⊞FUDNET_API void GatDoubleVar(char *Varname, double &Value, char *Viewout) { ... }
⊞FUDNET_API void GatIntVar(char *Varname, int &Value, char *Viewout) { ... }
⊞FUDNET_API void GatDoubleAry(char *Varname, double Value[], char *Viewout) { ... }
⊞FUDNET_API void GatIntAry(char *Varname, int Value[], char *Viewout) { ... }
⊞FUDNET_API void SatDoubleVar(char *Varname, double &Value, char *Viewout) { ... }
⊞FUDNET_API void SatIntVar(char *Varname, int &Value, char *Viewout) { ... }
⊞FUDNET_API void SatDoubleAry(char *Varname, double Value[], char *Viewout) { ... }
⊞FUDNET_API void SatIntAry(char *Varname, int Value[], char *Viewout) { ... }
```

3、从上到下分别是 double、int、double 数组、int 数组的值的获取（get）和输入（set）

4、Char *Varname 是变量名称，格式为“kakak_MOD2_gagag”,分别代表项目名称 kakak、模块号 MOD2、变量名称。Int &Value 是变量值的输出输入。Viewout 是此 MOD2 号对应的模块名，例如 pipe、valve 等管道阀门。

5、像变量名、项目名、模块名等，都是在模块编辑时就定义好的，流网中已经自动加载相关的定义文件：

6、此数据库速度：本地测速 800 万次读（写）/秒及以上，足够应对绝大多

数的数据通讯要求。

五、综上所述

1、概括一下调用的主要流程顺序是：运行主控程序、界面平台搭建、解析成数组或者 xml 文件输入主控程序、解析运行、界面平台读取数据以及输入参数改变。目前还没有涉及到工况保存的问题，需要与界面平台共同讨论如何进行工况保存，以确保兼容。